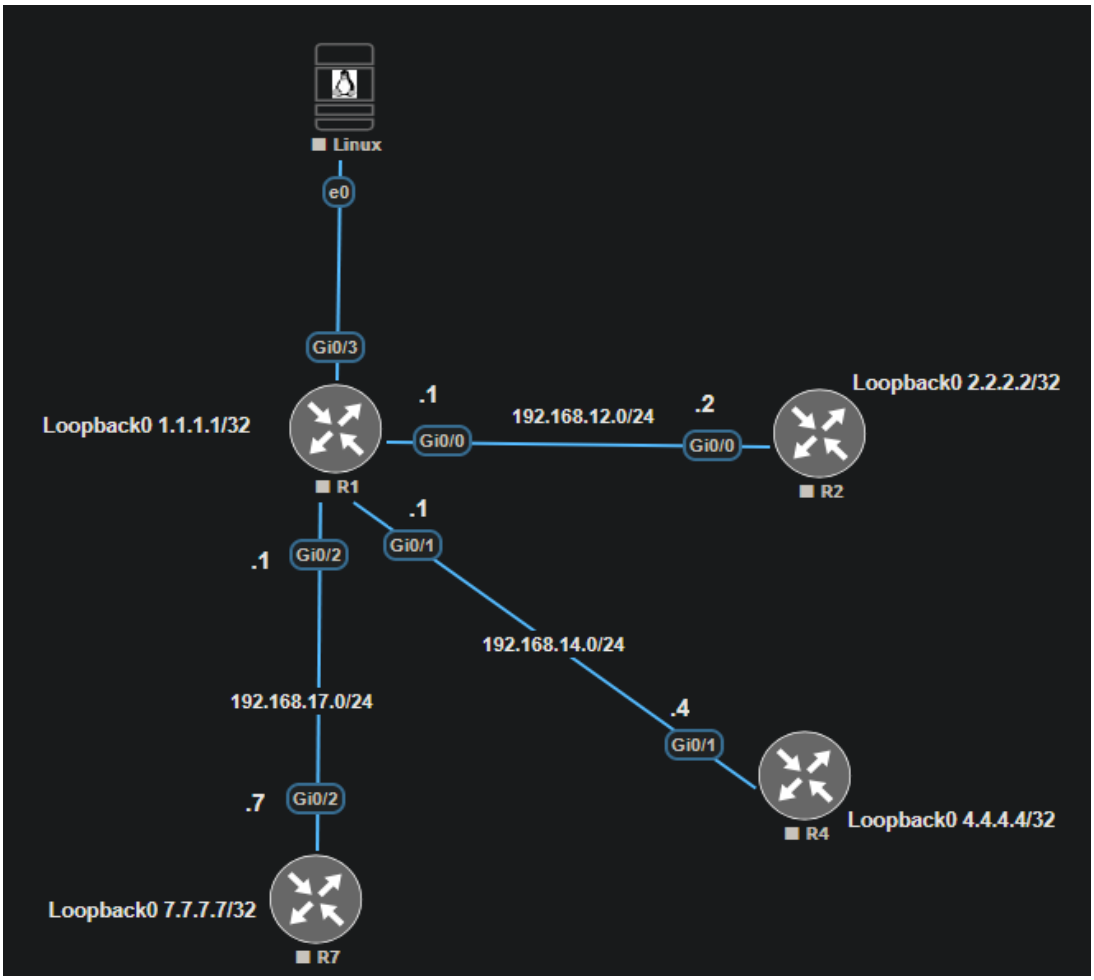
# CISCO IOS-XE LAB MANUAL

## Network Management: SNMPv2c, SNMPv3, Syslog & NetFlow

*EVE-NG Lab Environment | vIOS Topology R1-R2-R4-R7 with Linux SNMP/NMS Host*




### Topics Covered

SNMPv2c Community-Based Management • SNMPv3 USM Security • Syslog Centralized Logging • Traditional NetFlow • Flexible NetFlow

# Table of Contents

## 1. Lab Objectives & Overview

This lab manual builds on the previously established R1/R2/R4/R7 OSPF topology to implement enterprise network management services. By the end of this lab you will be able to configure, verify, and troubleshoot the following technologies on Cisco IOS-XE (vIOS):

- SNMPv2c community-based polling and trap notification
- SNMPv3 with authentication and privacy (USM security model)
- Centralized syslog logging from R4 to a Linux syslog server
- Traditional (classic) NetFlow export from R1
- Flexible NetFlow with custom flow records, exporters, and monitors on R1

### 1.1 Topology Summary

R1 is the hub router with four active interfaces. The Linux host on R1 Gi0/3 acts as the SNMP Network Management Station (NMS), the syslog collector, and the NetFlow collector for this lab.

| Device | Role | Loopback0 |
|---|---|---|
| R1 | Hub router / NetFlow exporter / SNMP + Syslog managed device | 1.1.1.1/32 |
| R2 | Spoke router / SNMP managed device | 2.2.2.2/32 |
| R4 | Spoke router / SNMP managed device / Syslog source | 4.4.4.4/32 |
| R7 | Spoke router / SNMP managed device | 7.7.7.7/32 |
| Linux | SNMP NMS, Syslog server, NetFlow collector | N/A - host on 192.168.100.0/24 |

> **NOTE: Base Configuration Assumed**
>
> This manual assumes interfaces, loopbacks, and OSPF (or equivalent full-reachability routing) from the prior lab series are already in place, and that every loopback address can already ping every other loopback address including the Linux host subnet. If reachability is not yet established, complete the OSPF multi-area lab manual first.

## 2. IP Addressing Reference

The Linux host subnet (192.168.100.0/24) is new for this lab and connects to R1 Gi0/3. Configure this interface and, if using OSPF, advertise it into the existing process (or use a static/default route) so all routers and the NMS have full reachability before proceeding.

| Device | Interface | IP Address | Connects To |
|---|---|---|---|
| R1 | Gi0/0 | 192.168.12.1/24 | R2 Gi0/0 |
| R1 | Gi0/1 | 192.168.14.1/24 | R4 Gi0/1 |
| R1 | Gi0/2 | 192.168.17.1/24 | R7 Gi0/2 |
| R1 | Gi0/3 | 192.168.100.1/24 | Linux e0 (NMS) |
| R1 | Loopback0 | 1.1.1.1/32 | - |
| R2 | Gi0/0 | 192.168.12.2/24 | R1 Gi0/0 |
| R2 | Loopback0 | 2.2.2.2/32 | - |
| R4 | Gi0/1 | 192.168.14.4/24 | R1 Gi0/1 |
| R4 | Loopback0 | 4.4.4.4/32 | - |
| R7 | Gi0/2 | 192.168.17.7/24 | R1 Gi0/2 |
| R7 | Loopback0 | 7.7.7.7/32 | - |
| Linux | e0 | 192.168.100.100/24 | R1 Gi0/3 |

> **TIP: Addressing Assumption**
>
> The Linux NMS address 192.168.100.100 is used consistently throughout every section of this manual as the SNMP trap host, the syslog host, and the NetFlow collector. Substitute your actual management IP if different.

# 3. SNMPv2c Configuration

## 3.1 SNMP Theory

SNMP (Simple Network Management Protocol) uses a manager/agent model. The NMS (manager) polls managed devices (agents) for values stored in the Management Information Base (MIB), addressed by Object Identifiers (OIDs). Agents can also proactively notify the manager of events using traps or informs.

| Operation | Direction | Purpose |
|---|---|---|
| GET | Manager → Agent | Retrieve the value of a single OID |
| GETNEXT | Manager → Agent | Walk the MIB tree one OID at a time |
| GETBULK | Manager → Agent | Retrieve multiple OIDs efficiently (v2c/v3 only) |
| SET | Manager → Agent | Modify a writable OID on the agent |
| TRAP | Agent → Manager | Unacknowledged, unsolicited event notification |
| INFORM | Agent → Manager | Acknowledged event notification (v2c/v3 only) |

### Version Comparison

| Feature | SNMPv1 | SNMPv2c | SNMPv3 |
|---|---|---|---|
| Authentication | Community string (clear text) | Community string (clear text) | Username + SHA/MD5 hash |
| Encryption | None | None | AES/DES (optional) |
| GETBULK | No | Yes | Yes |
| 64-bit counters | No | Yes | Yes |
| Security model | None | None | USM (User-based Security Model) |

> **WARNING: Cleartext Community Strings**
>
> SNMPv1 and SNMPv2c send the community string in plaintext on every packet. Anyone capturing traffic on the management VLAN can read it and gain read (or read-write) access to every device. Always pair v2c community strings with an ACL restricting which host may use them, and prefer SNMPv3 for anything beyond a lab environment (Section 4).

## 3.2 Configuration Plan

Configure all four routers (R1, R2, R4, R7) identically apart from IP addressing. Each router will:

- Restrict SNMP access to the Linux NMS (192.168.100.100) using a standard ACL

- Define a read-only community for polling and a read-write community for testing SET operations
- Send SNMPv2c traps to the NMS, sourced from Loopback0 for a stable source address
- Enable a representative set of trap types (linkup/linkdown, config change, cold/warm start)

### 3.3 R1 Configuration (Full Example)

```
R1(config)# ip access-list standard SNMP-NMS-ACL
R1(config-std-nacl)# permit host 192.168.100.100
R1(config-std-nacl)# deny any log
R1(config-std-nacl)# exit
!
R1(config)# snmp-server community NetLab-RO-2026 RO SNMP-NMS-ACL
R1(config)# snmp-server community NetLab-RW-2026 RW SNMP-NMS-ACL
!
R1(config)# snmp-server location EVE-NG-Lab-R1-HubRouter
R1(config)# snmp-server contact toran@netlab.local
!
R1(config)# snmp-server trap-source Loopback0
R1(config)# snmp-server host 192.168.100.100 version 2c NetLab-RO-2026
!
R1(config)# snmp-server enable traps snmp linkdown linkup coldstart warmstart
R1(config)# snmp-server enable traps config
R1(config)# snmp-server enable traps ospf state-change
```

Repeat the same block on R2, R4, and R7, changing only the snmp-server location string. The ACL, community strings, and trap host remain identical since they all point at the same NMS.

> **NOTE: Why Loopback0 as trap-source**
>
> snmp-server trap-source Loopback0 forces traps to always originate from the stable loopback address rather than whichever interface happens to route toward the NMS. This makes NMS-side filtering and correlation reliable even if a link goes down.

### 3.4 Linux NMS Configuration (net-snmp)

Install the net-snmp utilities and trap daemon on the Linux host:

```
$ sudo apt update
$ sudo apt install -y snmp snmp-mibs-downloader snmpd
$ sudo download-mibs
```

Configure the trap receiver to log incoming SNMPv2c traps to a dedicated file:

```
$ sudo tee /etc/snmp/snmptrapd.conf > /dev/null << 'EOF'
authCommunity log,execute,net NetLab-RO-2026
disableAuthorization yes
EOF

$ sudo systemctl stop snmpd
$ sudo snmptrapd -f -Lf /var/log/snmptrapd.log -c /etc/snmp/snmptrapd.conf udp:162
&
```

> **TIP: Persistent Service**
>
> On Debian/Ubuntu, the packaged snmptrapd.service ships without an [Install] section by design (trap listening is opt-in), so a plain sudo systemctl enable --now snmptrapd fails with "no installation config." To run it persistently across reboots, add the missing section yourself: sudo systemctl edit --full snmptrapd, add a [Install] block containing WantedBy=multi-user.target, then sudo systemctl daemon-reload && sudo systemctl enable --now snmptrapd. For a single lab session, running snmptrapd in the foreground (as shown above) or backgrounding it with & is sufficient.

### 3.5 Verification

From the Linux NMS, poll each router with snmpwalk/snmpget using the read-only community:

```
$ snmpget -v2c -c NetLab-RO-2026 192.168.14.4 sysDescr.0
$ snmpwalk -v2c -c NetLab-RO-2026 192.168.12.2 ifDescr
$ snmpwalk -v2c -c NetLab-RO-2026 1.1.1.1 system
```

Trigger a linkdown/linkup trap pair on R4 and confirm it arrives at the NMS:

```
R4(config)# interface Gi0/1
R4(config-if)# shutdown
R4(config-if)# no shutdown

$ tail -f /var/log/snmptrapd.log
```

On the router side, confirm the community strings and ACL binding, and check trap statistics:

```
R1# show snmp community
R1# show running-config | section snmp-server
R1# show snmp host
R1# show snmp
```

# 4. SNMPv3 Configuration

## 4.1 SNMPv3 Theory

SNMPv3 replaces the community string with the User-based Security Model (USM), which authenticates and optionally encrypts each SNMP packet on a per-user basis. Access is additionally restricted with the View-based Access Control Model (VACM), which maps users to groups, groups to security levels, and groups to MIB views.

| Security Level | Authentication | Encryption | IOS Keyword |
|---|---|---|---|
| noAuthNoPriv | None | None | noauth |
| authNoPriv | SHA or MD5 | None | auth |
| authPriv | SHA or MD5 | DES or AES | priv |

This lab uses the strongest level, authPriv, with SHA authentication and AES-128 encryption.

## 4.2 Configuration Plan

Configure the following on R1, R2, R4, and R7:

- An SNMP view limiting access to the standard MIB-II subtree
- A group that requires authPriv and binds to that view for both read and write
- A user in that group with a SHA auth password and an AES-128 privacy password
- An SNMPv3 trap host entry using the same ACL as Section 3

### 4.3 R1 Configuration (Full Example)

```
R1(config)# snmp-server view NETLAB-VIEW iso included
!
R1(config)# snmp-server group NETLAB-V3GROUP v3 priv read NETLAB-VIEW write
NETLAB-VIEW
R1(config)# snmp-server group NETLAB-V3GROUP v3 priv access SNMP-NMS-ACL
!
R1(config)# snmp-server user netlabadmin NETLAB-V3GROUP v3 auth sha AuthPass2026!
priv aes 128 PrivPass2026!
!
R1(config)# snmp-server host 192.168.100.100 version 3 priv netlabadmin
R1(config)# snmp-server trap-source Loopback0
```

Repeat on R2, R4, and R7 using the same view, group, user, and trap host. The SNMP-NMS-ACL from Section 3.3 is reused; no additional ACL configuration is required.

**IMPORTANT: Configuration Order Matters**

IOS-XE requires the view to exist before it is referenced by the group, and the group to exist before it is referenced by the user. Configure them in the order shown: view → group → user → host.

**WARNING: Passwords Shown Are Examples**

AuthPass2026! and PrivPass2026! are used here for readability. In any environment beyond an isolated lab, use unique, high-entropy passphrases and store them in a password manager, not in a document like this one.

### 4.4 Linux NMS: Polling with SNMPv3

net-snmp's command line tools accept SNMPv3 credentials directly on the command line:

```
$ snmpget -v3 -l authPriv -u netlabadmin \
    -a SHA -A AuthPass2026! \
    -x AES -X PrivPass2026! \
    192.168.14.4 sysUpTime.0

$ snmpwalk -v3 -l authPriv -u netlabadmin \
    -a SHA -A AuthPass2026! \
    -x AES -X PrivPass2026! \
    1.1.1.1 ifTable
```

Configure the SNMPv3 trap receiver by adding a createUser directive so snmptrapd can decrypt inbound traps, then restart the daemon:

```
$ sudo tee -a /var/lib/snmp/snmptrapd.conf > /dev/null << 'EOF'
createUser netlabadmin SHA AuthPass2026! AES PrivPass2026!
EOF

$ sudo tee -a /etc/snmp/snmptrapd.conf > /dev/null << 'EOF'
authUser log,execute,net netlabadmin priv
EOF

$ sudo systemctl restart snmptrapd
```

> **NOTE: Engine ID Persistence**
>
> SNMPv3 privacy keys are derived partly from the agent's engine ID. snmptrapd's createUser line must be loaded before the daemon starts listening, which is why it lives in the persistent-data file rather than being added live.

### 4.5 Verification

```
R1# show snmp user
R1# show snmp group
R1# show snmp view
R1# show snmp engineID
```

Generate a v3 trap by flapping an interface, then confirm decryption succeeded on the NMS (a failed auth/priv match will show as a discarded or unauthenticated packet rather than a decoded trap):

```
R4(config)# interface Gi0/1
R4(config-if)# shutdown
R4(config-if)# no shutdown

$ tail -f /var/log/snmptrapd.log
```

### 4.6 Common SNMPv3 Troubleshooting

| Symptom | Likely Cause | Fix |
|---|---|---|
| Timeout on snmpget/snmpwalk | ACL blocking NMS, or wrong security level flag | Verify SNMP-NMS-ACL permits 192.168.100.100; confirm -l authPriv matches the group |
| "Unknown Username" | User not yet configured, or typo in -u | show snmp user on the router; confirm exact username spelling |
| "Wrong Digest" / auth failure | Auth password mismatch or wrong hash algorithm (-a) | Re-enter snmp-server user with correct SHA password; confirm -a SHA matches router config |
| Trap arrives but shows garbled/undecoded data | snmptrapd missing createUser entry, or priv password mismatch | Add matching createUser line to snmptrapd.conf and restart the daemon |

# 5. Syslog: R4 to Linux NMS

## 5.1 Syslog Theory

Syslog is a message logging standard (RFC 5424) that decouples the software generating messages from the system that stores or analyzes them. Every message carries a facility (the type of process that generated it) and a severity (how urgent it is).

| Level | Keyword | Meaning |
|---|---|---|
| 0 | emergencies | System unusable |
| 1 | alerts | Immediate action needed |
| 2 | critical | Critical condition |
| 3 | errors | Error condition |
| 4 | warnings | Warning condition |
| 5 | notifications | Normal but significant |
| 6 | informational | Informational messages |
| 7 | debugging | Debug-level messages |

> **NOTE: This Lab's Target**
>
> R4 will be configured to send everything from informational (6) and more severe to the Linux collector, which is a common baseline that captures interface events and config changes without the volume of debug-level messages.

## 5.2 R4 Configuration

```
R4(config)# service timestamps log datetime msec localtime show-timezone
!
R4(config)# logging host 192.168.100.100 transport udp port 514
R4(config)# logging trap informational
R4(config)# logging source-interface Loopback0
R4(config)# logging facility local6
R4(config)# logging on
```

| Command | Purpose |
|---|---|
| service timestamps log datetime msec | Adds precise date/time to every logged message |
| logging host ... transport udp port 514 | Sends syslog messages to the Linux collector on standard UDP/514 |
| logging trap informational | Sets the minimum severity forwarded to the remote host (0-6) |
| logging source-interface Loopback0 | Sources syslog packets from the stable loopback, matching the SNMP/NetFlow pattern used elsewhere in this lab |

| | |
|---|---|
| logging facility local6 | Tags messages with a facility the receiving syslog server can filter on |

**TIP: Console/Buffer Logging Unaffected**

logging trap only controls what is forwarded to the remote syslog host. Console and buffered logging severities are configured separately with logging console and logging buffered if you want different thresholds locally.

## 5.3 Linux Collector Configuration (rsyslog)

Enable the UDP listener and create a dedicated log file for R4 so its messages are easy to isolate from other hosts:

```
$ sudo tee /etc/rsyslog.d/10-udp-listener.conf > /dev/null << 'EOF'
module(load="imudp")
input(type="imudp" port="514")
EOF

$ sudo tee /etc/rsyslog.d/20-r4.conf > /dev/null << 'EOF'
if $fromhost-ip == '192.168.14.4' then /var/log/r4.log
& stop
EOF

$ sudo systemctl restart rsyslog
```

> **WARNING: Firewall / Port 514**
>
> UDP/514 is a privileged port. Confirm rsyslog is actually listening (sudo ss -lun | grep 514) and that no host firewall (ufw/iptables) is blocking inbound UDP 514 from 192.168.14.4.

## 5.4 Verification

Generate a test message by making a small configuration change on R4, which triggers the standard %SYS-5-CONFIG_I message, then confirm it lands in the dedicated log file:

```
R4(config)# interface Loopback99
R4(config-if)# description SYSLOG-TEST
R4(config-if)# end

$ tail -f /var/log/r4.log
```

Expected output on Linux resembles:

```
Jul  2 09:41:02 192.168.14.4 4: *Jul  2 09:41:02.114: %SYS-5-CONFIG_I: Configured
from console by console
```

On R4, confirm the logging configuration and host reachability:

```
R4# show logging
R4# show logging | include Trap
```

> **NOTE: Clean Up Test Object**
>
> Remove the Loopback99 test interface once verification is complete: R4(config)# no interface Loopback99.

# 6. NetFlow & Flexible NetFlow on R1

## 6.1 NetFlow Theory

NetFlow tracks IP traffic as it transits an interface, grouping packets into flows by a key (traditionally the 7-tuple: source/destination IP, source/destination port, protocol, ToS, and input interface) and periodically exporting flow records to a collector. R1, as the hub router, sees traffic to and from every spoke, making it the ideal export point.

| Aspect | Traditional (Classic) NetFlow | Flexible NetFlow (FNF) |
|---|---|---|
| Key fields | Fixed 7-tuple | Fully customizable via flow record |
| Configuration objects | ip flow ingress/egress on the interface | flow record, flow exporter, flow monitor, then applied to the interface |
| Export versions | v5, v9 | v9, IPFIX |
| Non-IP flow tracking (e.g. MAC, MPLS) | No | Yes |
| Typical use case | Quick, simple visibility | Custom fields, multiple simultaneous monitors |

## 6.2 Design: Splitting Interfaces Between the Two Methods

R1 has four active interfaces. Rather than mixing both flow technologies on the same interface (which can double-count traffic and complicates the collector output), this lab applies each technology to a distinct pair of interfaces:

| Interface | Neighbor | Flow Technology Applied |
|---|---|---|
| Gi0/0 | R2 | Traditional NetFlow (ip flow ingress/egress) |
| Gi0/1 | R4 | Traditional NetFlow (ip flow ingress/egress) |
| Gi0/2 | R7 | Flexible NetFlow (flow monitor, input/output) |
| Gi0/3 | Linux NMS | Flexible NetFlow (flow monitor, input/output) |

> **TIP: Why Split Rather Than Overlap**
>
> This layout lets you compare both technologies side by side on the same router and collector, which is useful for learning purposes. In production, a single router typically standardizes on one method — usually Flexible NetFlow, since classic NetFlow is legacy on current IOS-XE releases.

## 6.3 Traditional NetFlow — Gi0/0 and Gi0/1

```
R1(config)# ip flow-export version 9
R1(config)# ip flow-export destination 192.168.100.100 9996
R1(config)# ip flow-export source Loopback0
!
R1(config)# ip flow-cache timeout active 1
R1(config)# ip flow-cache timeout inactive 15
!
R1(config)# interface GigabitEthernet0/0
R1(config-if)# ip flow ingress
R1(config-if)# ip flow egress
R1(config-if)# exit
!
R1(config)# interface GigabitEthernet0/1
R1(config-if)# ip flow ingress
R1(config-if)# ip flow egress
R1(config-if)# exit
```

| Command | Purpose |
|---|---|
| ip flow-export version 9 | Uses NetFlow v9, which supports templates and is compatible with most modern collectors |
| ip flow-export destination ... 9996 | Sends export packets to the Linux collector on the conventional NetFlow UDP port 9996 |
| ip flow-cache timeout active 1 | Forces long-lived flows to export every 1 minute instead of the 30-minute default, useful for fast lab verification |
| ip flow ingress / egress | Enables flow accounting for traffic entering / leaving the interface |

## 6.4 Flexible NetFlow — Gi0/2 and Gi0/3

Flexible NetFlow separates the flow definition (record), the destination (exporter), and the cache behavior (monitor) into independent, reusable objects.

### Step 1: Flow Record

```
R1(config)# flow record NETLAB-RECORD-V4
R1(config-flow-record)# description IPv4 flow record for R1 south interfaces
R1(config-flow-record)# match ipv4 source address
R1(config-flow-record)# match ipv4 destination address
R1(config-flow-record)# match ipv4 protocol
R1(config-flow-record)# match transport source-port
R1(config-flow-record)# match transport destination-port
R1(config-flow-record)# match interface input
R1(config-flow-record)# match ipv4 tos
R1(config-flow-record)# collect interface output
R1(config-flow-record)# collect counter bytes long
R1(config-flow-record)# collect counter packets long
```

```
R1(config-flow-record)# collect timestamp sys-uptime first
R1(config-flow-record)# collect timestamp sys-uptime last
```

### Step 2: Flow Exporter

```
R1(config)# flow exporter NETLAB-EXPORTER
R1(config-flow-exporter)# description Export to Linux NMS collector
R1(config-flow-exporter)# destination 192.168.100.100
R1(config-flow-exporter)# transport udp 9996
R1(config-flow-exporter)# source Loopback0
R1(config-flow-exporter)# export-protocol netflow-v9
R1(config-flow-exporter)# template data timeout 60
```

### Step 3: Flow Monitor

```
R1(config)# flow monitor NETLAB-MONITOR
R1(config-flow-monitor)# description R1 south-side flexible netflow monitor
R1(config-flow-monitor)# record NETLAB-RECORD-V4
R1(config-flow-monitor)# exporter NETLAB-EXPORTER
R1(config-flow-monitor)# cache timeout active 60
R1(config-flow-monitor)# cache timeout inactive 15
```

### Step 4: Apply to Interfaces

```
R1(config)# interface GigabitEthernet0/2
R1(config-if)# ip flow monitor NETLAB-MONITOR input
R1(config-if)# ip flow monitor NETLAB-MONITOR output
R1(config-if)# exit
!
R1(config)# interface GigabitEthernet0/3
R1(config-if)# ip flow monitor NETLAB-MONITOR input
R1(config-if)# ip flow monitor NETLAB-MONITOR output
R1(config-if)# exit
```

> **NOTE: Same Collector, Same Port**
>
> Both the classic export (Section 6.3) and the Flexible NetFlow exporter target 192.168.100.100:9996 using NetFlow v9. The collector distinguishes flows by their source router/interface metadata inside each record; a single nfcapd instance can receive both.

## 6.5 Linux Collector Configuration (nfdump / nfcapd)

```
$ sudo apt install -y nfdump
$ sudo mkdir -p /var/nfcapd/r1
$ sudo nfcapd -D -w /var/nfcapd/r1 -p 9996 -S 1
```

| Flag | Meaning |
|---|---|
| -D | Fork to background (daemonize) |
| -w | Output directory to store captured flow files (the target directory must already exist) |
| -p | UDP port to listen on (9996, matching both exporters) |
| -S 1 | Adds a numeric sub-directory hierarchy under the flowdir for easier retention management |

> **WARNING: nfdump Version Differences**
>
> Older nfdump releases (1.6.x) used -l basedir to set the output directory and treated -w as a separate sync flag. Current nfdump (1.7.x, as shipped on Ubuntu 24.04/Debian 12) merged this into -w flowdir and dropped -l entirely. Run nfcapd -h to confirm which syntax your installed version expects before relying on the command above.

For persistence across reboots, wrap this in a systemd unit rather than running it ad hoc.

## 6.6 Verification

### Traditional NetFlow (Gi0/0, Gi0/1)

```
R1# show ip flow interface
R1# show ip cache flow
R1# show ip flow export
```

### Flexible NetFlow (Gi0/2, Gi0/3)

```
R1# show flow record NETLAB-RECORD-V4
R1# show flow exporter NETLAB-EXPORTER
R1# show flow exporter statistics
R1# show flow monitor NETLAB-MONITOR
R1# show flow monitor NETLAB-MONITOR cache format table
```

### On the Linux Collector

Generate traffic across all four interfaces (ping/traceroute from R2, R4, R7, and the Linux host toward each other's loopbacks), then read the captured flow files:

```
$ nfdump -R /var/nfcapd/r1 -o extended | head -30
$ nfdump -R /var/nfcapd/r1 -s ip/flows -n 10
$ nfdump -R /var/nfcapd/r1 'src ip 192.168.14.4'
```

**TIP: Confirm Both Sources Appear**

You should see flows tagged with input interface Gi0/0 or Gi0/1 (from the classic NetFlow cache) alongside flows tagged Gi0/2 or Gi0/3 (from the Flexible NetFlow monitor) in the same nfdump output, confirming both exporters are reaching the collector.

# 7. Consolidated Verification Checklist

Use this checklist to confirm every service in this lab is functioning end to end before moving on.

| # | Check | Command / Action | Expected Result |
|---|---|---|---|
| 1 | SNMPv2c polling | snmpwalk -v2c -c NetLab-RO-2026 <router> | Returns MIB data, no timeout |
| 2 | SNMPv2c trap | Flap an interface on R4 | Trap appears in /var/log/snmptrapd.log |
| 3 | SNMPv3 polling | snmpget -v3 -l authPriv -u netlabadmin ... | Returns MIB data, no auth error |
| 4 | SNMPv3 trap | Flap an interface on R4 | Decrypted trap appears in /var/log/snmptrapd.log |
| 5 | Syslog | Make a config change on R4 | Message appears in /var/log/r4.log within seconds |
| 6 | Classic NetFlow | show ip cache flow on R1 | Active flows listed for Gi0/0 and Gi0/1 |
| 7 | Flexible NetFlow | show flow monitor NETLAB-MONITOR cache | Active flows listed for Gi0/2 and Gi0/3 |
| 8 | NetFlow collector | nfdump -R /var/nfcapd/r1 | Flow records visible from all four R1 interfaces |

# 8. Review Questions

Answer the following without referring back to the configuration sections. Suggested answers are not included in this manual so they can be used for self-assessment or peer review.

**1.** What is the fundamental security weakness of SNMPv1 and SNMPv2c, and which SNMPv3 security model addresses it?

**2.** Name the three SNMPv3 security levels and state which one this lab used, along with its authentication and privacy protocols.

**3.** In IOS-XE SNMPv3 configuration, why must the view be created before the group, and the group before the user?

**4.** What is the purpose of snmp-server trap-source, and why was Loopback0 chosen for it in this lab?

**5.** Explain the difference between an SNMP TRAP and an SNMP INFORM. Which one is acknowledged by the receiver?

**6.** Which SNMP operation allows a manager to retrieve multiple OIDs in a single request, and in which SNMP versions is it available?

**7.** What command restricts which hosts may use a given SNMP community string on an IOS-XE router?

**8.** List the eight syslog severity levels from most to least severe.

**9.** Which IOS-XE command controls the minimum severity of messages forwarded to a remote syslog server, and what value was used for R4 in this lab?

**10.** Why was logging source-interface Loopback0 configured on R4 rather than letting the router choose an interface automatically?

**11.** On the Linux side, what rsyslog module must be loaded to accept syslog messages sent over UDP?

**12.** Describe the difference between traditional (classic) NetFlow and Flexible NetFlow in terms of how the flow key is defined.

**13.** What three configuration objects make up a Flexible NetFlow deployment, and what does each one control?

**14.** Why did this lab apply classic NetFlow and Flexible NetFlow to different interfaces on R1 rather than combining them on the same interface?

**15.** What NetFlow export versions were used in this lab, and which UDP port did both exporters send to?

**16.** What is the effect of lowering ip flow-cache timeout active from its default (30 minutes) to 1 minute, and why is this useful in a lab setting?

**17.** In the Flexible NetFlow flow record NETLAB-RECORD-V4, which fields are match fields (part of the flow key) and which are collect fields (informational only)?

**18.** What is the purpose of the -S 1 flag when starting nfcapd on the Linux collector?

**19.** If snmpwalk from the Linux NMS times out against a router, list at least three possible causes and how you would check each one.

**20.** If a Flexible NetFlow flow monitor shows zero flows in its cache despite active traffic, what two places would you check first on the router?

## Appendix A: R1 Consolidated Configuration

The complete set of SNMP, and NetFlow related configuration lines added to R1 in this lab, consolidated for reference.

```
! --- ACL used for both SNMPv2c and SNMPv3 ---
ip access-list standard SNMP-NMS-ACL
 permit host 192.168.100.100
 deny any log
!
! --- SNMPv2c ---
snmp-server community NetLab-RO-2026 RO SNMP-NMS-ACL
snmp-server community NetLab-RW-2026 RW SNMP-NMS-ACL
snmp-server location EVE-NG-Lab-R1-HubRouter
snmp-server contact toran@netlab.local
snmp-server trap-source Loopback0
snmp-server host 192.168.100.100 version 2c NetLab-RO-2026
snmp-server enable traps snmp linkdown linkup coldstart warmstart
snmp-server enable traps config
snmp-server enable traps ospf state-change
!
! --- SNMPv3 ---
snmp-server view NETLAB-VIEW iso included
snmp-server group NETLAB-V3GROUP v3 priv read NETLAB-VIEW write NETLAB-VIEW
snmp-server group NETLAB-V3GROUP v3 priv access SNMP-NMS-ACL
snmp-server user netlabadmin NETLAB-V3GROUP v3 auth sha AuthPass2026! priv aes 128
PrivPass2026!
snmp-server host 192.168.100.100 version 3 priv netlabadmin
!
! --- Traditional NetFlow (Gi0/0, Gi0/1) ---
ip flow-export version 9
ip flow-export destination 192.168.100.100 9996
ip flow-export source Loopback0
ip flow-cache timeout active 1
ip flow-cache timeout inactive 15
!
interface GigabitEthernet0/0
 ip flow ingress
 ip flow egress
!
interface GigabitEthernet0/1
 ip flow ingress
 ip flow egress
!
! --- Flexible NetFlow (Gi0/2, Gi0/3) ---
flow record NETLAB-RECORD-V4
 match ipv4 source address
 match ipv4 destination address
 match ipv4 protocol
```

```
 match transport source-port
 match transport destination-port
 match interface input
 match ipv4 tos
 collect interface output
 collect counter bytes long
 collect counter packets long
 collect timestamp sys-uptime first
 collect timestamp sys-uptime last
!
flow exporter NETLAB-EXPORTER
 destination 192.168.100.100
 source Loopback0
 transport udp 9996
 export-protocol netflow-v9
 template data timeout 60
!
flow monitor NETLAB-MONITOR
 record NETLAB-RECORD-V4
 exporter NETLAB-EXPORTER
 cache timeout active 60
 cache timeout inactive 15
!
interface GigabitEthernet0/2
 ip flow monitor NETLAB-MONITOR input
 ip flow monitor NETLAB-MONITOR output
!
interface GigabitEthernet0/3
 ip flow monitor NETLAB-MONITOR input
 ip flow monitor NETLAB-MONITOR output
```

# Appendix B: R4 Configuration & Linux Cheat Sheet

## R4 Consolidated Configuration

```
! --- SNMPv2c / v3 (same pattern as R1, different location string) ---
ip access-list standard SNMP-NMS-ACL
 permit host 192.168.100.100
 deny any log
!
snmp-server community NetLab-RO-2026 RO SNMP-NMS-ACL
snmp-server community NetLab-RW-2026 RW SNMP-NMS-ACL
snmp-server location EVE-NG-Lab-R4-Spoke
snmp-server contact toran@netlab.local
snmp-server trap-source Loopback0
snmp-server host 192.168.100.100 version 2c NetLab-RO-2026
snmp-server view NETLAB-VIEW iso included
snmp-server group NETLAB-V3GROUP v3 priv read NETLAB-VIEW write NETLAB-VIEW
snmp-server group NETLAB-V3GROUP v3 priv access SNMP-NMS-ACL
snmp-server user netlabadmin NETLAB-V3GROUP v3 auth sha AuthPass2026! priv aes 128
PrivPass2026!
snmp-server host 192.168.100.100 version 3 priv netlabadmin
!
! --- Syslog ---
service timestamps log datetime msec localtime show-timezone
logging host 192.168.100.100 transport udp port 514
logging trap informational
logging source-interface Loopback0
logging facility local6
logging on
```

## Linux Command Reference

| Purpose | Command |
|---|---|
| Install SNMP tools | sudo apt install -y snmp snmp-mibs-downloader snmpd |
| Poll v2c | snmpwalk -v2c -c NetLab-RO-2026 <ip> <oid> |
| Poll v3 | snmpget -v3 -l authPriv -u netlabadmin -a SHA -A AuthPass2026! -x AES -X PrivPass2026! <ip> <oid> |
| Start trap receiver | sudo snmptrapd -f -Lf /var/log/snmptrapd.log -c /etc/snmp/snmptrapd.conf udp:162 |
| Watch syslog from R4 | tail -f /var/log/r4.log |
| Check rsyslog listening | sudo ss -lun \| grep 514 |
| Install NetFlow collector | sudo apt install -y nfdump |
| Start nfcapd | sudo nfcapd -D -w /var/nfcapd/r1 -p 9996 -S 1 |

| Purpose | Command |
|---|---|
| Read flow files | nfdump -R /var/nfcapd/r1 -o extended |